

Sisteme de Operare

Cap 8. Probleme de sincronizare. Deadlocks.

April 11, 2023

- 1 Probleme clasice de sincronizare
- 2 Primitive de sincronizare din kernel
- 3 Primitive de sincronizare POSIX
- 4 Abordări alternative
- 5 Deadlocks

- 1 Probleme clasice de sincronizare
- 2 Primitive de sincronizare din kernel
- 3 Primitive de sincronizare POSIX
- 4 Abordări alternative
- 5 Deadlocks

Problema producător-consumator (bounded buffer)

- problemele clasice de sincronizare sunt folosite pentru a testa fiecare schemă de sincronizare propusă
- atât producătorul cât și consumatorul accesează un buffer limitat la N locații
- ambii au acces la următoarele structuri de date:

```
int n;
```

```
semaphore mutex = 1;
```

```
semaphore empty = n;
```

```
semaphore full = 0;
```

- semaforul binar `mutex` furnizează acces exclusiv la buffer și este inițializat cu 1
- semaforul `empty` numără bufferele libere, iar `full` pe cele ocupate; inițial avem doar N buffere libere și 0 ocupate
- semafoarele vor bloca, după caz, fie producătorul (s-au umplut toate bufferele, `empty` ajunge la 0, sau consumatorul (s-au golit toate bufferele, `full` ajunge la 0)

Problema producător-consumator (2)

- thread-ul producător:

```
while (true) {
    ...
    // produce an item
    // in next_produced
    ...
    wait(empty);
    wait(mutex);
    ...
    // add next_produced
    // to the buffer
    ...
    signal(mutex);
    signal(full);
}
```

- thread-ul consumator:

```
while (true) {
    wait(full);
    wait(mutex);
    ...
    // remove an item from
    // buffer to next_consumed
    ...
    signal(mutex);
    signal(empty);
    ...
    // consume the item
    // in next_consumed
    ...
}
```

Problema cititorilor și scriitorilor

- problemă clasică ce modelează accesul la o bază de date, unde mai multe procese vor să scrie concomitent cu altele ce vor să citească
- riscurile se pot elimina prin accesarea bazei de date în mod exclusiv; există mai multe variante
- **prima variantă**: un cititor nu trebuie să fie pus să aștepte decât dacă există un scriitor care a obținut acces exclusiv la bază; nici un cititor nu va aștepta după un scriitor care așteaptă
- **a doua variantă**: un scriitor care este pregătit va face scrierea cel mai devreme posibil; adică un scriitor așteaptă să scrie, nici unui cititor nu îi va fi permisă citirea
- o soluție la oricare dintre probleme duce la **starvation**: în primul caz pentru scriitori, în cel de-al doilea, pentru cititori
- soluția pentru prima variantă folosește următoarele structuri de date:

```
semaphore rw_mutex = 1;
semaphore mutex = 1;
int read count = 0;
```

Problema cititorilor și scriitorilor (2)

- thread-ul cititor:

```
while (true) {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);
    ...
    /* reading is performed */
    ...
    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
}
```

- thread-ul scriitor:

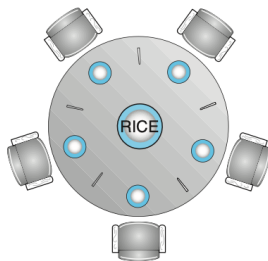
```
while (true) {
    wait(rw_mutex);
    ...
    /* writing is performed */
    ...
    signal(rw_mutex);
}
```

- semaforul binar mutex este folosit pentru accesul exclusiv la variabila read_count
- read_count numără câți cititori citesc concurrent

Problema cititorilor și scriitorilor (3)

- `rw_mutex` funcționează ca un mutex pentru scriitori, permițând accesul doar a unui singur scriitor pentru scriere, și asigurând că nici un cititor nu citește simultan cu scrierea
- dacă un scriitor scrie în secțiunea critică și sunt N cititori care așteaptă, atunci unul singur așteaptă după `rw_mutex` și $N - 1$ așteaptă după mutex
- după instrucțiunea `signal(rw_mutex)` se reia execuția fie a unui scriitor fie a primului cititor care a vrut să citească în timp ce un scriitor scria
- problema a fost rezolvată cu ajutorul lock-urilor de scriere sau citire: un proces ce modifică datele cere un lock de scriere, cel care doar citește, un lock de citire
- lock-urile de citire și scriere sunt utile:
 - în aplicațiile unde este ușor de identificat ce procese fac doar modificare respectiv procese care doar citesc;
 - în aplicații unde avem mult mai mulți cititori decât scriitori; lock-urile de citire-scriere necesită mai multe resurse decât semafoarele

Problema cinei filozofilor



- sunt 5 filozofi în jurul unei mese pe care se află un bol cu orez
- fiecare filozof are în dreapta și-n stînga sa câte un bețișor; ca să mănînce are nevoie de cele două bețișoare din dreapta și din stînga sa
- fiecare filozof poate să ia la un moment dat doar un bețișor și nu poate lua un bețișor din mîna vecinului
- este un exemplu reprezentativ pentru o clasă largă de situații de concurență; exemplu de alocare a resurselor între procese fără starvation și fără deadlocks

Cina filozofilor: soluția cu semafoare

- soluția simplă este să reprezentăm fiecare bețișor cu un semafor binar, semaphore chopstick[5], inițializat cu un vector de valori de 1

```
while (true) {
    wait(chopstick[i]);
    wait(chopstick[(i+1) % 5]);
    ...
    /* eat for a while */
    ...
    signal(chopstick[i]);
    signal(chopstick[(i+1) % 5]);
    ...
    /* think for awhile */
    ...
}
```

- deși soluția garantează că doi vecini nu pot mânca simultan, ea generează deadlock
- scenariu: fiecare filozof ridică simultan bețișorul din dreapta, și se blochează așteptând să-l poată lua și pe cel din stânga

Cina filozofilor: soluții

- propuneri de remediere a deadlock-ului:
 - limităm numărul de filozofi ce pot sta simultan la masă la maxim 4;
 - permitem ca un filozof să poată lua bețișoarele doar dacă ambele sunt disponibile;
 - folosirea unei soluții asimetrice: filozofii cu număr impar ridică întâi furculița din stânga și apoi pe cea din dreapta, în vreme ce cei cu număr par procedează invers.
- o soluție ar fi că dacă un filozof găsește unul din bețișoare ocupate, să-l lase jos și pe celălalt și să încerce din nou → starvation
- **soluția cu monitor** presupune ca filozoful să ia bețișoarele doar dacă ambele sunt disponibile
- viața unui filozof se rezumă la trei stări:


```
enum {THINKING, HUNGRY, EATING} state[5];
```
- un filozof i își poate seta starea $state[i] = EATING$ doar dacă vecinii săi nu mănâncă:


```
state[(i+4) % 5] != EATING &&  
state[(i+1) % 5] != EATING
```

Cina filozofilor: soluția cu monitor

```

monitor DiningPhilosophers
{
    enum {THINKING, HUNGRY,
          EATING} state[5];
    condition self[5];
    void pickup(int i) {
        state[i] = HUNGRY;
        test(i);
        if (state[i] != EATING)
            self[i].wait();
    }
    void putdown(int i) {
        state[i] = THINKING;
        test((i + 4) % 5);
        test((i + 1) % 5);
    }
}

```

```

void test(int i) {
    if ((state[(i + 4) % 5] !=
        EATING) &&
        (state[i] == HUNGRY) &&
        (state[(i + 1) % 5] !=
        EATING)) {
        state[i] = EATING;
        self[i].signal();
    }
}

initialization code() {
    for (int i = 0; i < 5; i++)
        state[i] = THINKING;
}
}

```

Cina filozofilor: descrierea soluției cu monitor

- fiecare filozof trebuie să invoce `pickup()` înainte de-a mânca, ceea ce se poate solda cu suspendarea procesului; după ce mănâncă va chema `putdown()`, ceea ce poate duce la "trezirea" vecinilor
- filozofii vor fi construiți din secvența:

```
DiningPhilosophers.pickup(i);  
...  
// eat  
...  
DiningPhilosophers.putdown(i);
```

- soluția nu permite ca doi vecini să mănânce simultan și nu are deadlocks, dar e posibil ca un filozof să flămânzească

- 1 Probleme clasice de sincronizare
- 2 Primitive de sincronizare din kernel
- 3 Primitive de sincronizare POSIX
- 4 Abordări alternative
- 5 Deadlocks

Sincronizarea în Windows

- când SO accesează o resursă globală pentru un sistem cu un singur procesor, el maschează întreruperile pentru apelurile sistem care ar putea accesa aceeași resursă
- SO folosește **obiecte dispatcher** pentru sincronizarea thread-urilor folosind semafoare, mutex-uri, evenimente sau timers
- dispatcher-ele pot fi într-o stare **signalled**, în care sunt disponibile, iar un thread care preia acel obiect dispatch nu se va bloca, respectiv **non-signalled**, la preluarea obiectului thread-ul se va bloca, va intra în coada "waiting"
- un obiect de tip **secțiune critică** poate fi folosit de un thread fără intervenția kernel-ului; aceasta se face prin utilizarea unui spinlock; dacă durează prea mult, se va apela la kernel care va alocă un mutex, eliberând CPU-ul; utile deoarece în practică factorul de **contention** este suficient de redus

Sincronizarea în Linux

- până la versiunea 2.6 kernel-ul era non-preemptiv, adică un proces într-un apel de sistem nu putea fi evacuat în coada de "ready"
- cea mai simplă primitivă de sincronizare este un întreg atomic:

```
atomic_t counter;
int value;
```

<i>Atomic Operation</i>	<i>Effect</i>
<code>atomic_set(&counter, 5);</code>	<code>counter = 5</code>
<code>atomic_add(10, &counter);</code>	<code>counter = counter + 10</code>
<code>atomic_sub(4, &counter);</code>	<code>counter = counter - 4</code>
<code>atomic_inc(&counter);</code>	<code>counter = counter + 1</code>
<code>value = atomic_read(&counter);</code>	<code>value = 12</code>

- folosite pentru scenarii de variabile partajate, care nu necesită mecanisme de locking costisitoare
- pentru folosirea mutex-urilor, un task va apela funcțiile `mutex_lock()`, ce poate suspenda procesul, respectiv `mutex_unlock()`

Sincronizarea în Linux (2)

Single Processor	Multiple Processors
Disable kernel preemption	Acquire spin lock
Enable kernel preemption	Release spin lock

- pe sistemele SMP, spinlock-urile sunt folosite ca mecanisme de sincronizare, ele sunt deținute pe termen scurt
- pe sistemele uni-procesor, kernel-ul dezactivează preemptia
- atât spinlock-urile cât și mutex lock-urile sunt **nerecursive**, adică dacă un thread a obținut acel lock, nu îl poate obține din nou fără ca în prealabil să elibereze acel lock
- un task Linux are asociată o structură `thread_info` ce conține un counter, `preempt_count`, ce numără câte lock-uri au fost obținute de un task; cât timp acest counter nu este zero, `preemption`-ul nu poate fi activat, ca atare task-ul curent trebuie să termine apelul de sistem

- 1 Probleme clasice de sincronizare
- 2 Primitive de sincronizare din kernel
- 3 Primitive de sincronizare POSIX**
- 4 Abordări alternative
- 5 Deadlocks

Mutex-uri POSIX

- primitive care sunt disponibile programatorului de aplicații (dacă SO implementează specificația POSIX)
- pthread-urile folosesc tipul de dată `pthread_mutex_t` pentru mutex-uri; crearea și inițializarea sa:

```
#include <pthread.h>
pthread_mutex_t mutex;
pthread_mutex_init(&mutex, NULL);
```

```
/* acquire the mutex lock */
pthread_mutex_lock(&mutex);
/* critical section */
/* release the mutex lock */
pthread_mutex_unlock(&mutex);
```

- funcțiile întorc 0 pentru succes, sau diferită de zero în caz de eroare

Semafoare POSIX

- sunt parte a extensiei POSIX SEM; ele pot fi **named** (cu nume asociat) sau **unnamed** (anonime):

```
#include <semaphore.h>
sem_t *sem;
/* create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

- flag-ul `O_CREAT` creează semaforul dacă el nu există deja; la el au drept de acces toate procesele (masca 0666 în baza 8)
- avantajul definirii unui nume este acela că procesele care vor deschide un semafor cu acel nume și aceiași parametri vor obține un descriptor ce indică spre semaforul deja creat
- apelurile de blocare respectiv deblocare a unui semafor sunt `sem_wait(sem)` respectiv `sem_post(sem)`, apeluri care bordează secțiunea critică

Semafoare POSIX anonime

```
#include <semaphore.h>
sem_t sem;
/* create the semaphore and initialize it to 1 */
sem_init(&sem, 0, 1);
```

- primul parametru este pointer-ul la semafor, al doilea nivelul de partajare (sharing) iar al treilea este valoarea semaforului
- valoarea de sharing 0 indică faptul că semaforul poate fi partajat doar cu thread-urile care fac parte din procesul care a inițializat semaforul
- se folosesc aceleași primitive de sincronizare `sem_wait(sem)` și `sem_post(sem)`

Variabile de condiție POSIX

- variabilele de condiție nu sunt folosite aici în cadrul unui monitor, ci asociate cu un mutex

```
pthread_mutex_t mutex;  
pthread_cond_t cond_var;
```

```
pthread_mutex_init(&mutex, NULL);  
pthread_cond_init(&cond_var, NULL);
```

- iată cum un thread poate aștepta îndeplinirea unei condiții:

```
pthread_mutex_lock(&mutex);  
while (a != b)  
    pthread_cond_wait(&cond_var, &mutex);
```

```
pthread_mutex_unlock(&mutex);
```

- mutex-ul trebuie să fie blocat înainte de apelul `pthread_cond_wait`, deoarece protejează datele din test de posibile condiții de concurență

Variabile de condiție POSIX (2)

- apelul `pthread_cond_wait` va elibera mutex-ul, permițînd accesul altor thread-uri
- verificarea condiției trebuie plasată într-o buclă `while` pentru a preveni erorile de programare, astfel ea se re-verifică înainte de părăsirea secțiunii critice - este posibil ca programatorul să ridice semnalul chiar dacă acea condiție nu e îndeplinită

```
pthread_mutex_lock(&mutex);  
a = b;  
pthread_cond_signal(&cond_var);  
pthread_mutex_unlock(&mutex);
```

- `pthread_cond_signal` nu eliberează mutex-ul, ci instrucțiunea următoare
- după eliberarea mutex-ului thread-ul blocat poate prelua mutex-ul și re-testa condiția

- 1 Probleme clasice de sincronizare
- 2 Primitive de sincronizare din kernel
- 3 Primitive de sincronizare POSIX
- 4 Abordări alternative**
- 5 Deadlocks

Memoria tranzacțională

- pe măsură ce procesoarele SMP sunt tot mai răspândite, devine dificil de a preveni condițiile de concurență și deadlock-ul doar cu primitivele clasice (mutex, semafoare, monitoare, variabile de condiție)
- ideea de **memorie tranzacțională** vine din bazele de date; se efectuează o secvență de scrieri și citiri, secvență care este atomică (indivizibilă): dacă toate operațiile reușesc, tranzacția se comite; altfel, starea de dinaintea tranzacției este refăcută ("rolled-back")
- abordarea clasică cu obținerea secțiunii critice folosind `acquire()` urmat de `release()` poate duce la deadlock-uri sau alte probleme; o capabilitate care folosește posibilitățile memoriei tranzacționale este cuvântul cheie `atomic` care este urmat de un bloc de cod:

```
void update () {  
    atomic {  
        /* modify shared data */  
    }  
}
```

Memoria tranzacțională (2)

- avantajul este că sistemul de memorie tranzacțională (și nu programatorul) este responsabil de garantarea atomicității
- pentru că nu sunt implicate mutex-uri sau semafoare, nu se poate ajunge la deadlock
- **software transactional memory** (STM) implementează sistemul exclusiv în software; inserează cod specific în blocurile tranzacțiilor, cu ajutorul unui compilator; se examinează ce instrucțiuni pot rula în paralel și unde este necesar locking-ul low-level
- **hardware transactional memory** (HTM) folosește ierarhia de cache precum și protocoalele de menținerea coerenței cache-ului pentru a rezolva conflictele apărute între memoriile cache ale diferitelor procesoare
- HTM nu necesită o instrumentare specifică a codului și are overhead mai redus ca STM; necesită însă modificarea protocoalelor de coerență a cache-ului pentru a suporta memoria tranzacțională

OpenMP

- orice instrucțiune C (sau bloc de instrucțiuni) ce urmează unei directive `#pragma omp parallel` va fi executată în paralel pe un număr de thread-uri egal cu numărul de core-uri din sistem
- restricționarea prin serializarea accesului thread-urilor la un set de instrucțiuni se face cu directiva `#pragma omp critical`, un singur thread poate fi activ în acea secțiune critică la un moment dat

```
void update(int value) {  
    #pragma omp critical  
    {  
        counter += value;  
    }  
}
```

- previne apariția condițiilor de concurență dar nu ne scutește de apariția deadlock-urilor

Limbaje de programare funcționale

- majoritatea limbajelor cunoscute - C, C++, Java, C# - sunt limbaje **imperative** (procedurale); ele sunt folosite pentru implementarea de algoritmi bazați pe stare
- starea unui algoritm este reprezentată de variabile și diverse structuri de date; starea se poate schimba pe parcursul execuției algoritmului
- limbajele de programare **funcționale** urmează o paradigmă diferită; limbajele funcționale nu păstrează starea, adică o variabilă odată inițializată, ea devine **immutable** (nu se poate schimba)
- din cauza variabilelor immutable, problemele de concurență specificate anterior nu pot exista în limbajele funcționale (ex. Erlang, Scala)

- 1 Probleme clasice de sincronizare
- 2 Primitive de sincronizare din kernel
- 3 Primitive de sincronizare POSIX
- 4 Abordări alternative
- 5 Deadlocks

Modelul sistemului

- un număr de resurse finite sunt distribuite unui număr de procese ce concurează pentru aceste resurse
- resursele precum mutex-urile și semafoarele sunt cele mai comune cauze ale deadlock-urilor
- SO păstrează o tabelă cu evidența resurselor alocate și libere; tabela conține, pentru fiecare resursă alocată, thread-ul care deține temporar acea resursă
- exemplu: problema cinei filozofilor, simultan fiecare ia bețișorul din stânga
- dezvoltatorul trebuie să fie atent la modul de obținere și eliberare al acestor resurse ce previn condițiile de concurență

Deadlock-uri în aplicații multi-threaded

```

/* thread one */                                /* thread two runs in this func
void *do_work_one(void *param) void *do_work_two(void *param)
{
    pthread_mutex_lock(&mtx_1);                pthread_mutex_lock(&mtx_2);
    pthread_mutex_lock(&mtx_2);                pthread_mutex_lock(&mtx_1);
    /**                                         /**
    * Do some work                             * Do some work
    */                                         */
    pthread_mutex_unlock(&mtx_2);                pthread_mutex_unlock(&mtx_1);
    pthread_mutex_unlock(&mtx_1);                pthread_mutex_unlock(&mtx_2);
    pthread_exit(0);                            pthread_exit(0);
}                                              }

```

- deadlock-ul poate să nu apară, dar se poate manifesta dacă thread-urile se sincronizează perfect - este dificil de prevăzut un deadlock și dificil de testat dacă pot să apară

Livelock

```
/* thread_one runs in this function */
```

```
void *do_work_one(void *param)
```

```
{
    int done = 0;

    while (!done) {
        pthread_mutex_lock(&first_mutex);
        if (pthread_mutex_trylock(&second_mutex)
            /**
             * Do some work
             */
            pthread_mutex_unlock(&second_mutex);
            pthread_mutex_unlock(&first_mutex);
            done = 1;
        }
        else
            pthread_mutex_unlock(&first_mutex);
    }

    pthread_exit(0);
}
```

```
/* thread_two runs in this function */
```

```
void *do_work_two(void *param)
```

```
{
    int done = 0;

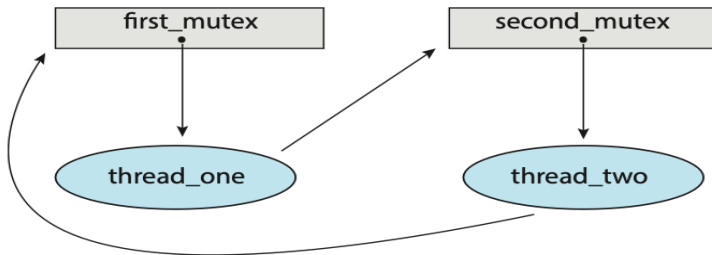
    while (!done) {
        pthread_mutex_lock(&second_mutex);
        if (pthread_mutex_trylock(&first_mutex)) {
            /**
             * Do some work
             */
            pthread_mutex_unlock(&first_mutex);
            pthread_mutex_unlock(&second_mutex);
            done = 1;
        }
        else
            pthread_mutex_unlock(&second_mutex);
    }

    pthread_exit(0);
}
```

- apare de regulă când thread-urile reîncearcă simultan operația; poate fi evitată dacă fiecare thread reîncearcă operația la intervale random (algoritmul random backoff al protocolului Ethernet)

Condițiile necesare simultan pentru un deadlock

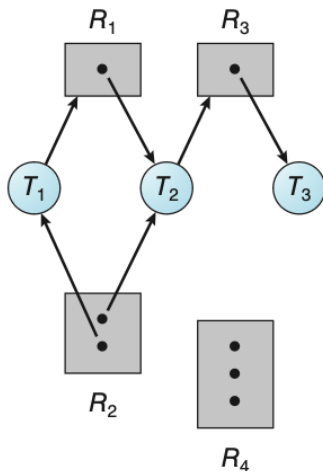
- 1 **excluderea mutuală**: doar un thread poate utiliza resursa la un moment dat;
- 2 **hold and wait**: un thread a obținut deja o resursă blocabilă și așteaptă eliberarea celei de-a doua;
- 3 **no preemption**: resursa poate fi eliberată doar voluntar de thread-ul ce o deține;
- 4 **așteptare circulară**: există un set de thread-uri $T_0, T_1, \dots, T_n$ în așa fel încât T_0 așteaptă o resursă blocată de T_1 , T_1 așteaptă o resursă blocată de T_2 , ș.a.m.d. până la T_n așteaptă o resursă blocată de T_0 ,



Graful de alocare a resurselor

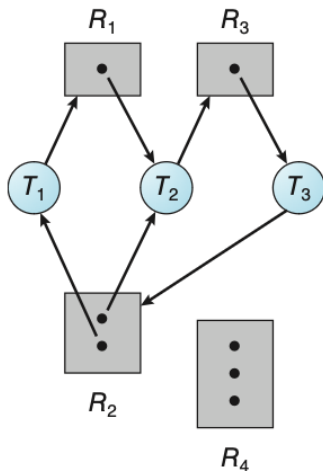
- deadlock-urile se pot reprezenta adecvat cu ajutorul unui graf orientat
- graful constă într-un set de vârfuri V și un set de muchii E
- vârfurile (nodurile) pot fi de două feluri: thread-urile active $T = T_1, T_2, \dots, T_n$ și resursele $R = R_1, R_2, \dots, R_m$
- arcul orientat de la task-ul i la resursa j , $T_i \rightarrow R_j$ este un **request edge**, de cerere de obținere a resursei (apel lock), iar cel orientat de la resursa j la task-ul i , $R_j \rightarrow T_i$ este un **assignment edge**, de alocare a resursei (revenire din apelul lock)
- task-ul este reprezentat printr-un dreptunghi iar resursa printr-un cerc (elipsă)
- la apariția unei cereri de resursă se introduce un arc (coardă) $T_i \rightarrow R_j$; la satisfacerea cererii, arcul se **înlocuiește** cu arcul invers $R_j \rightarrow T_i$

Exemplu de graf de alocare a resurselor



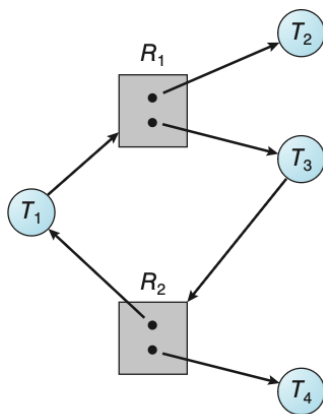
- resursele R_1 și R_3 au câte o singură instanță; resursa R_2 are 2 instanțe iar R_4 3 instanțe
- T_1 are o cerere către R_2 a cărei instanță e deținută de T_2 ; T_1 și T_2 dețin fiecare câte o instanță din R_2
- T_2 are o cerere către R_3 a cărei instanță este deținută de T_3
- nu există deadlock
- dacă fiecare resursă are doar o instanță și putem forma un ciclu, atunci a apărut un deadlock; existența unui ciclu nu înseamnă neapărat ciclu pentru resurse cu mai multe instanțe

Exemplu de graf de alocare cu deadlock



- există două cicluri în sistem:
 - 1 $T_1 \rightarrow R_1 \rightarrow T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1$
 - 2 $T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_2 \rightarrow T_2$
- thread-urile T_1 , T_2 , T_3 sunt deadlocked (blocate)

Exemplu de graf de alocare fără deadlock



- există un ciclu
 $T_1 \rightarrow R_1 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1$
- nu avem deadlock: task-ul T_4 va elibera resursa R_2 și aceasta va putea fi preluată de T_3 , terminând ciclul
- dacă nu avem cicluri, atunci sistemul nu este blocat; dacă avem cicluri, sistemul **poate** fi blocat, dar nu e obligatoriu

Prevenirea deadlock-ului

- **excluderea mutuală**: se evită folosirea excluderii mutuale, de exemplu prin marcarea resursei ca read-only (fișier); cererile de deschidere simultană read-only sunt satisfăcute
- **hold and wait**: pentru evitarea sa, impunem ca un thread care cere o resursă să nu mai dețină blocată nici o altă resursă
- **no preemption**: dacă un thread deține niște resurse și cere o resursă ce este ocupată, atunci toate resursele deținute sunt eliberate
- **așteptarea circulară**: pentru a preveni formarea unui ciclu putem impune o ordonare a resurselor, adică dacă un thread deține resurse cu un identificator N , poate cere resurse doar cu identificator strict mai mare ca N